

Cyberster Internship Program

Internship Report

BLUE TEAM

MUHAMMAD RAZA

ROLL NUMBER: CSI-B1-121

WEEK 1: LAB SETUP & SOC FOUNDATIONS

Overview

During the first week of my SOC internship, the primary objective was to build a defensive cybersecurity laboratory environment from scratch and understand the foundational mechanics of network monitoring and SIEM architecture. The tasks progressed from analyzing fundamental network traffic to deploying a fully functional Wazuh SIEM, connecting endpoint agents, and ultimately triaging live security alerts.

The lab consists of three virtual machines:

- **Kali Linux:** Attacker / Penetration Testing Machine
- **Ubuntu Server:** Wazuh Manager / SIEM
- **Windows 10:** Monitored Endpoint

DAY 1: NETWORKING FUNDAMENTALS FOR SOC

Task 01: OSI Model, TCP/IP, and Core Networking

Before analyzing live traffic or deploying virtual machines, it was critical to establish a foundational understanding of how network communication operates.

The OSI & TCP/IP Models: Networking communication is structured in layers. The **OSI Model** breaks this down into 7 layers: Physical, Data Link (MAC addresses/switches), Network (IP addresses/routing), Transport (TCP/UDP), Session, Presentation, and Application (HTTP/DNS). The **TCP/IP Model** is the practical implementation used on the internet, condensing this into 4 layers: Network Access, Internet, Transport, and Application. Understanding these layers allows a SOC analyst to isolate where an anomaly is occurring (e.g., a routing issue at the Network layer vs. a malicious payload at the Application layer).

Subnetting & IP Addressing: Subnetting involves dividing a large network into smaller, isolated, and more manageable networks. Using CIDR notation (like a /24 subnet mask), we can define the boundary between the network identifier and the host identifier. This is crucial for SOC analysts to determine if an IP address is communicating internally on a local network or reaching out to an external, potentially malicious, IP.

Common Ports for SOC Monitoring: Analysts frequently monitor specific ports for expected behavior. Anomalies on these ports often indicate compromise:

- **Port 22 (SSH):** Secure remote login.
- **Port 80 (HTTP) & 443 (HTTPS):** Standard web traffic.
- **Port 53 (DNS):** Domain name resolution.
- **Port 445 (SMB):** Windows file sharing (heavily monitored for lateral movement).
- **Port 3389 (RDP):** Remote Desktop Protocol.

Task 02: Protocol Analysis & Wireshark Capture

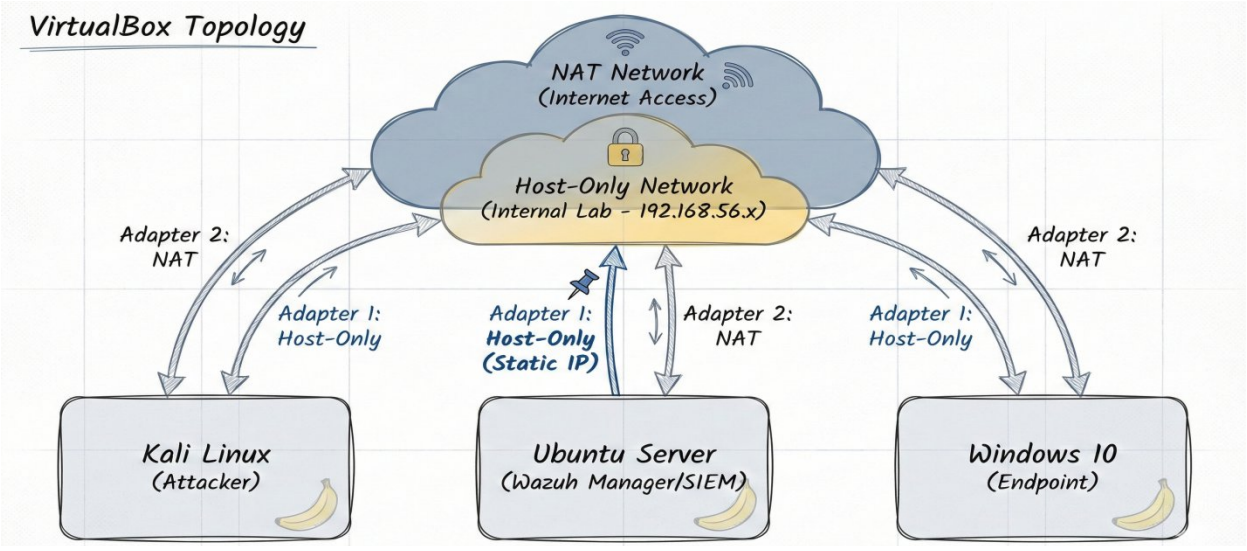
Using Wireshark, I captured live traffic to observe how everyday browsing translates into packet exchanges on the wire. Identifying normal traffic behavior is the first step in spotting anomalies.

Below is a summary of the core protocols analyzed during the capture:

- **HTTP:** Transmits data in cleartext. In a SOC context, unexpected HTTP traffic on non-standard ports often indicates command-and-control (C2) beacons.
- **HTTPS/TLS:** Encrypts web traffic. While the payload is secure, the TLS handshake reveals metadata such as the Server Name Indication (SNI), identifying the destination domain.
- **ARP:** Translates logical IP addresses into physical MAC addresses on a local network. A suspicious volume of ARP replies without corresponding requests can indicate ARP spoofing.
- **DNS:** Translates domain names into IP addresses. Anomalies such as massive TXT records or frequent queries to random subdomains may indicate DNS tunneling or data exfiltration.
- **ICMP:** Used for network diagnostics (e.g., ping). Suspicious ICMP activity might include abnormally large packet sizes or a high frequency of pings sweeping across a subnet.

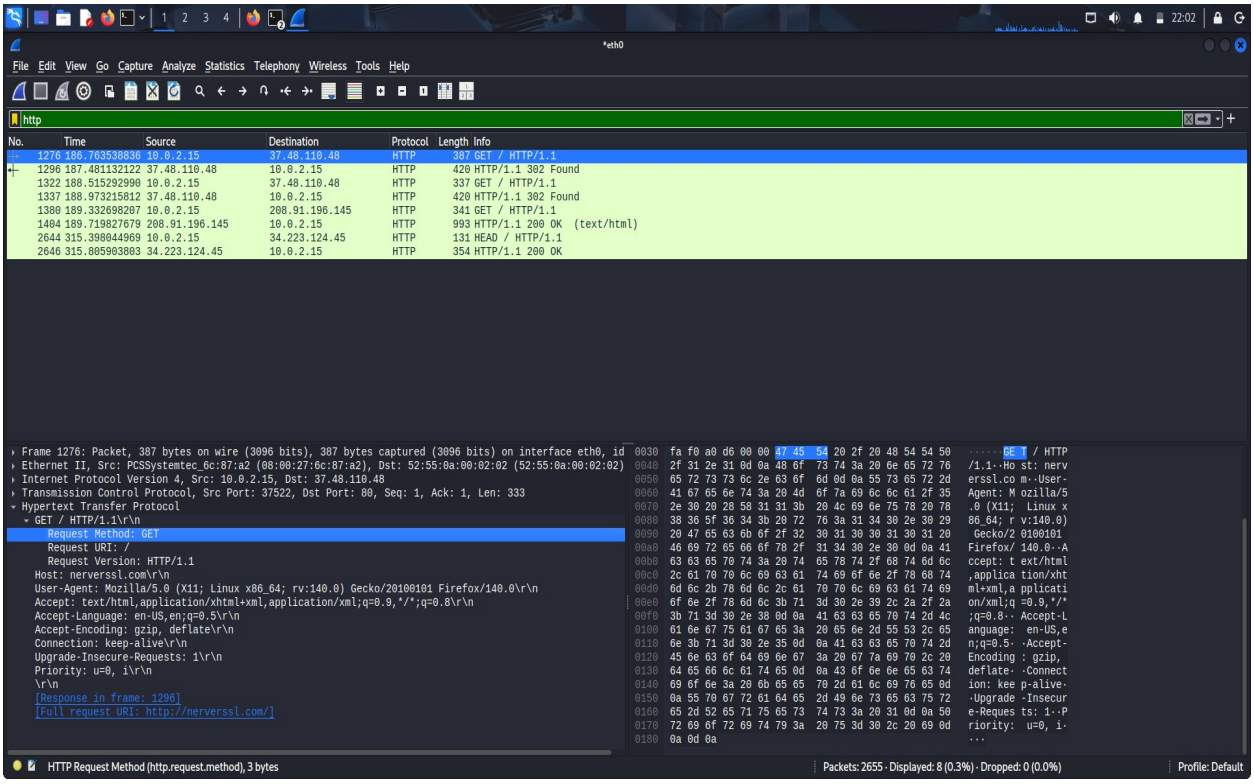
Network Topology Diagram

I designed a secure architecture for the lab using VirtualBox, utilizing dual network adapters for each machine to ensure internal isolation while maintaining internet access for updates.

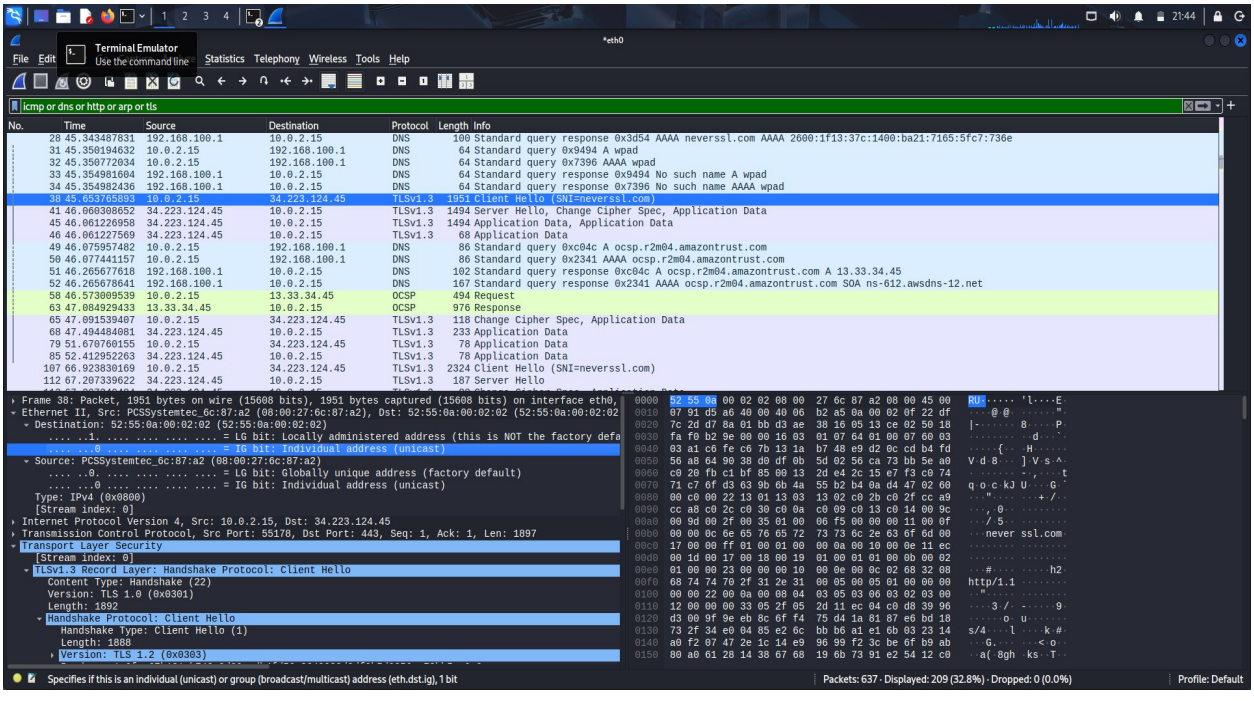


Protocol Captures

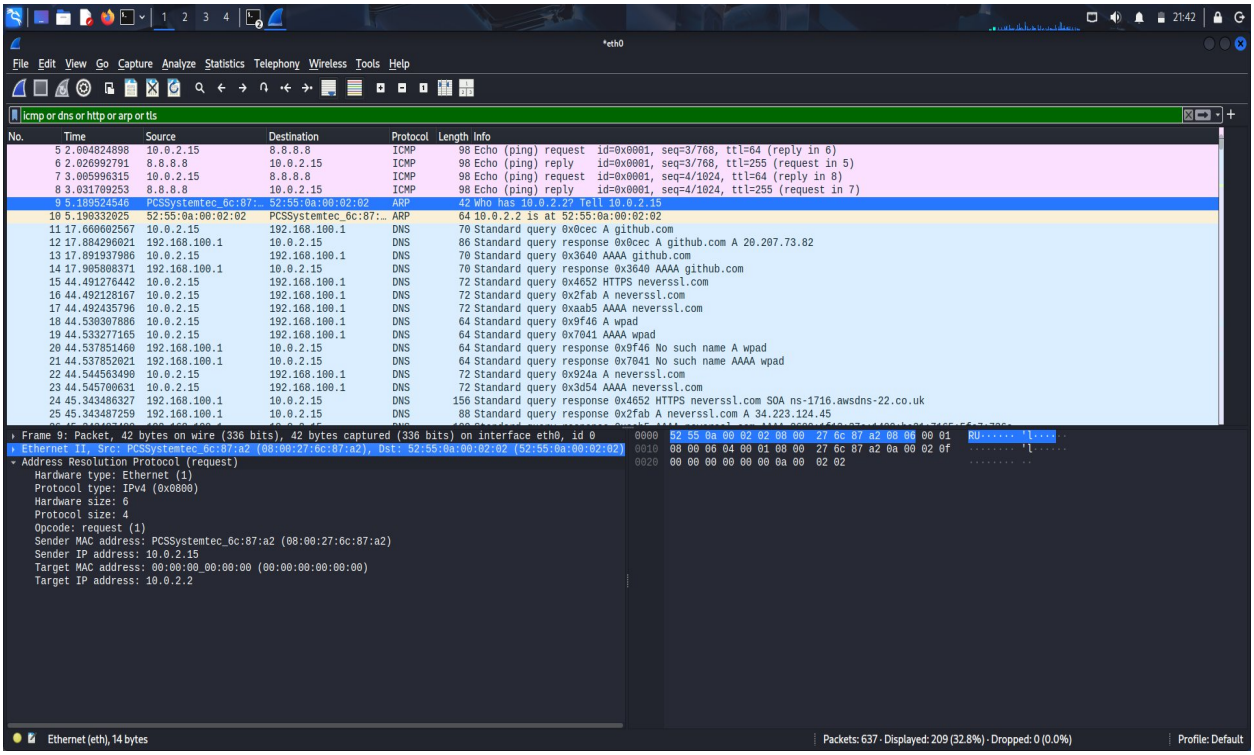
HTTP



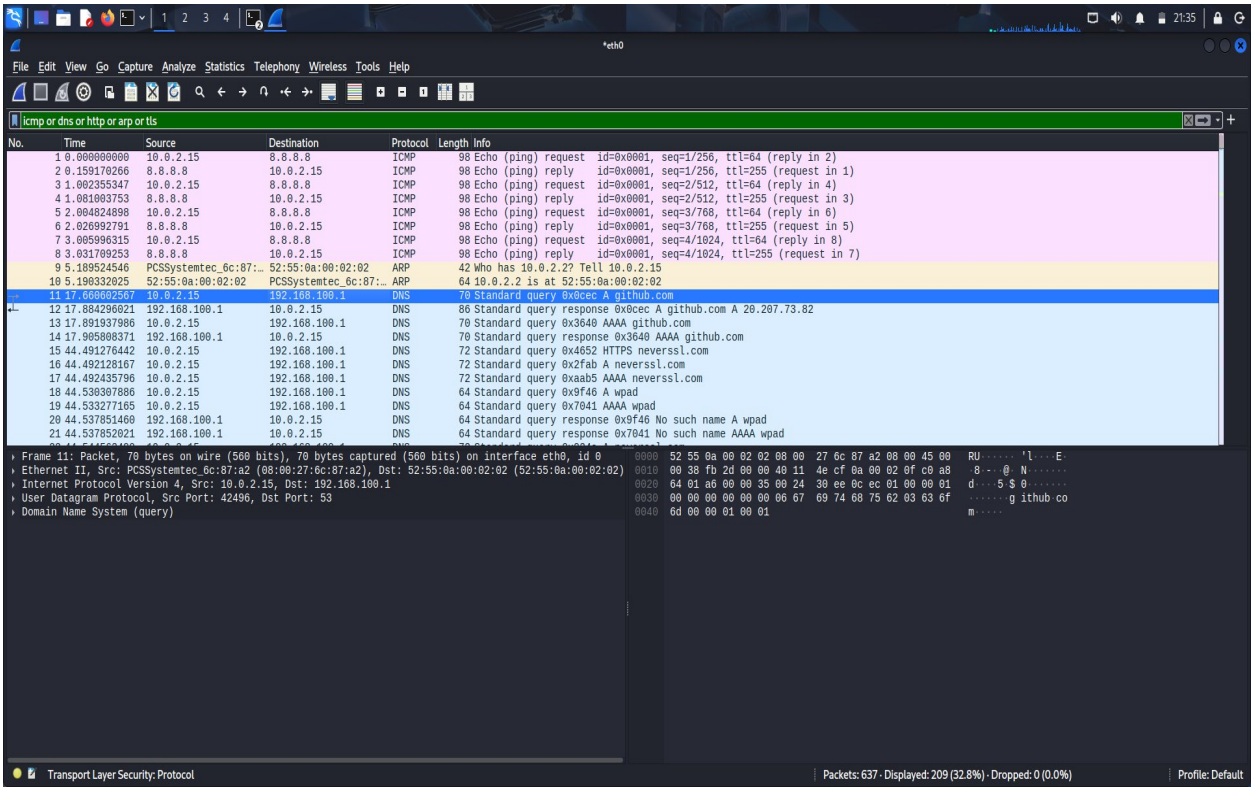
TLS



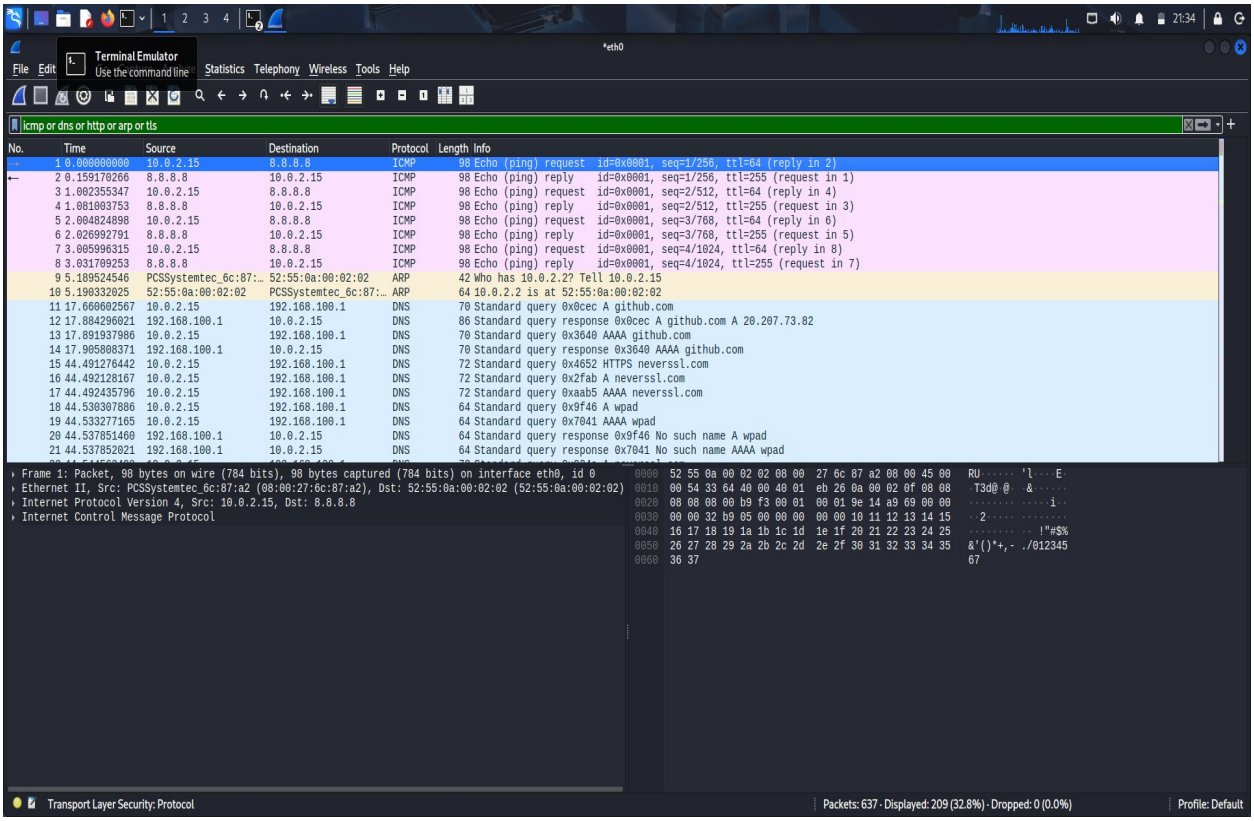
ARP


*eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
icmp or dns or http or arp or tls
No. Time Source Destination Protocol Length Info
5 2.004824898 10.0.2.15 8.8.8.8 ICMP 98 Echo (ping) request id=0x0001, seq=3/768, ttl=64 (reply in 6)
6 2.026992791 8.8.8.8 10.0.2.15 ICMP 98 Echo (ping) reply id=0x0001, seq=3/768, ttl=255 (request in 5)
7 3.005996315 10.0.2.15 8.8.8.8 ICMP 98 Echo (ping) request id=0x0001, seq=4/1024, ttl=64 (reply in 8)
8 3.031709253 8.8.8.8 10.0.2.15 ICMP 98 Echo (ping) reply id=0x0001, seq=4/1024, ttl=255 (request in 7)
9 5.189524546 PCSSystemtec_6c:87:... 52:55:0a:00:02:02 ARP 42 Who has 10.0.2.2? Tell 10.0.2.15
10 5.190332025 52:55:0a:00:02:02 PCSSystemtec_6c:87:... ARP 64 10.0.2.2 is at 52:55:0a:00:02:02
11 17.660602567 10.0.2.15 192.168.100.1 DNS 70 Standard query 0x0cec A github.com
12 17.884296021 192.168.100.1 10.0.2.15 DNS 86 Standard query response 0x0cec A github.com A 20.207.73.82
13 17.891937986 10.0.2.15 192.168.100.1 DNS 70 Standard query 0x3640 AAAA github.com
14 17.905808371 192.168.100.1 10.0.2.15 DNS 70 Standard query response 0x3640 AAAA github.com
15 44.491276442 10.0.2.15 192.168.100.1 DNS 72 Standard query 0x4652 HTTPS neverssl.com
16 44.492128167 10.0.2.15 192.168.100.1 DNS 72 Standard query 0x2fab A neverssl.com
17 44.492435796 10.0.2.15 192.168.100.1 DNS 72 Standard query 0xaab5 AAAA neverssl.com
18 44.530307886 10.0.2.15 192.168.100.1 DNS 64 Standard query 0x9f46 A wpad
19 44.533277165 10.0.2.15 192.168.100.1 DNS 64 Standard query 0x7041 AAAA wpad
20 44.537851460 192.168.100.1 10.0.2.15 DNS 64 Standard query response 0x9f46 No such name A wpad
21 44.537852021 192.168.100.1 10.0.2.15 DNS 64 Standard query response 0x7041 No such name AAAA wpad
22 44.544563490 10.0.2.15 192.168.100.1 DNS 72 Standard query 0x924a A neverssl.com
23 44.545700631 10.0.2.15 192.168.100.1 DNS 72 Standard query 0x3d54 AAAA neverssl.com
24 45.343486327 192.168.100.1 10.0.2.15 DNS 156 Standard query response 0x4652 HTTPS neverssl.com SOA ns-1716.awsdns-22.co.uk
25 45.343487259 192.168.100.1 10.0.2.15 DNS 88 Standard query response 0x2fab A neverssl.com A 34.223.124.45
Frame 9: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface eth0, id 0
Ethernet II, Src: PCSSystemtec_6c:87:a2 (08:00:27:6c:87:a2), Dst: 52:55:0a:00:02:02 (52:55:0a:00:02:02)
Address Resolution Protocol (request)
Hardware type: Ethernet (1)
Protocol type: IPv4 (0x0800)
Hardware size: 6
Protocol size: 4
Opcode: request (1)
Sender MAC address: PCSSystemtec_6c:87:a2 (08:00:27:6c:87:a2)
Sender IP address: 10.0.2.15
Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
Target IP address: 10.0.2.2
0000 52 55 0a 00 02 02 08 00 27 6c 87 a2 08 06 00 01
0010 08 00 06 04 00 01 08 00 27 6c 87 a2 0a 00 02 0f
0020 00 00 00 00 00 00 0a 00 02 02
Ethernet (eth), 14 bytes
Packets: 637 · Displayed: 209 (32.8%) · Dropped: 0 (0.0%)
Profile: Default

DNS


*eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
icmp or dns or http or arp or tls
No. Time Source Destination Protocol Length Info
1 0.000000000 10.0.2.15 8.8.8.8 ICMP 98 Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 2)
2 0.159170266 8.8.8.8 10.0.2.15 ICMP 98 Echo (ping) reply id=0x0001, seq=1/256, ttl=255 (request in 1)
3 1.002355347 10.0.2.15 8.8.8.8 ICMP 98 Echo (ping) request id=0x0001, seq=2/512, ttl=64 (reply in 4)
4 1.081003753 8.8.8.8 10.0.2.15 ICMP 98 Echo (ping) reply id=0x0001, seq=2/512, ttl=255 (request in 3)
5 2.004824898 10.0.2.15 8.8.8.8 ICMP 98 Echo (ping) request id=0x0001, seq=3/768, ttl=64 (reply in 6)
6 2.026992791 8.8.8.8 10.0.2.15 ICMP 98 Echo (ping) reply id=0x0001, seq=3/768, ttl=255 (request in 5)
7 3.005996315 10.0.2.15 8.8.8.8 ICMP 98 Echo (ping) request id=0x0001, seq=4/1024, ttl=64 (reply in 8)
8 3.031709253 8.8.8.8 10.0.2.15 ICMP 98 Echo (ping) reply id=0x0001, seq=4/1024, ttl=255 (request in 7)
9 5.189524546 PCSSystemtec_6c:87:... 52:55:0a:00:02:02 ARP 42 Who has 10.0.2.2? Tell 10.0.2.15
10 5.190332025 52:55:0a:00:02:02 PCSSystemtec_6c:87:... ARP 64 10.0.2.2 is at 52:55:0a:00:02:02
11 17.660602567 10.0.2.15 192.168.100.1 DNS 70 Standard query 0x0cec A github.com
12 17.884296021 192.168.100.1 10.0.2.15 DNS 86 Standard query response 0x0cec A github.com A 20.207.73.82
13 17.891937986 10.0.2.15 192.168.100.1 DNS 70 Standard query 0x3640 AAAA github.com
14 17.905808371 192.168.100.1 10.0.2.15 DNS 70 Standard query response 0x3640 AAAA github.com
15 44.491276442 10.0.2.15 192.168.100.1 DNS 72 Standard query 0x4652 HTTPS neverssl.com
16 44.492128167 10.0.2.15 192.168.100.1 DNS 72 Standard query 0x2fab A neverssl.com
17 44.492435796 10.0.2.15 192.168.100.1 DNS 72 Standard query 0xaab5 AAAA neverssl.com
18 44.530307886 10.0.2.15 192.168.100.1 DNS 64 Standard query 0x9f46 A wpad
19 44.533277165 10.0.2.15 192.168.100.1 DNS 64 Standard query 0x7041 AAAA wpad
20 44.537851460 192.168.100.1 10.0.2.15 DNS 64 Standard query response 0x9f46 No such name A wpad
21 44.537852021 192.168.100.1 10.0.2.15 DNS 64 Standard query response 0x7041 No such name AAAA wpad
Frame 11: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface eth0, id 0
Ethernet II, Src: PCSSystemtec_6c:87:a2 (08:00:27:6c:87:a2), Dst: 52:55:0a:00:02:02 (52:55:0a:00:02:02)
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 192.168.100.1
User Datagram Protocol, Src Port: 42496, Dst Port: 53
Domain Name System (query)
0000 52 55 0a 00 02 02 08 00 27 6c 87 a2 08 00 45 00
0010 00 38 fb 2d 00 00 40 11 4e cf 0a 00 02 0f c0 a8
0020 64 01 a6 00 00 35 00 24 30 ee 0c ec 01 00 00 01
0030 00 00 00 00 00 00 06 67 69 74 68 75 62 03 63 6f
0040 6d 00 00 01 00 01
Transport Layer Security: Protocol
Packets: 637 · Displayed: 209 (32.8%) · Dropped: 0 (0.0%)
Profile: Default

ICMP



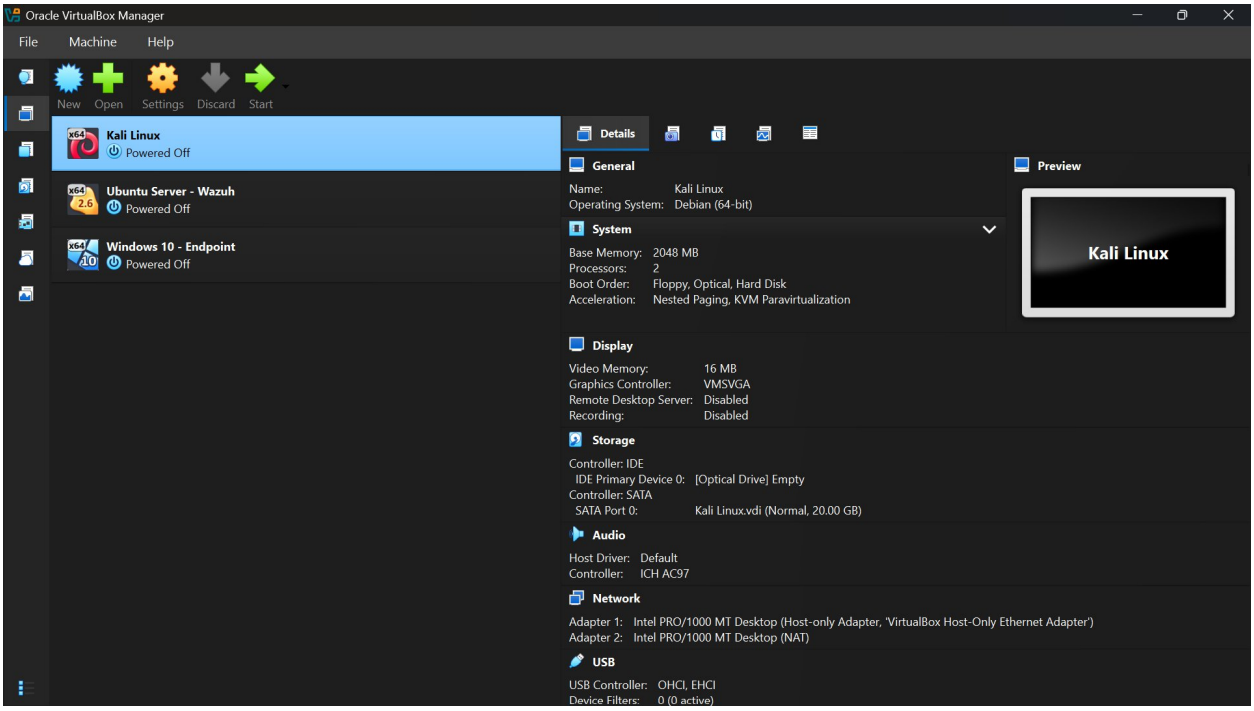
DAY 2: VIRTUALBOX LAB SETUP

Task 01: Virtual Machine & Network Configuration

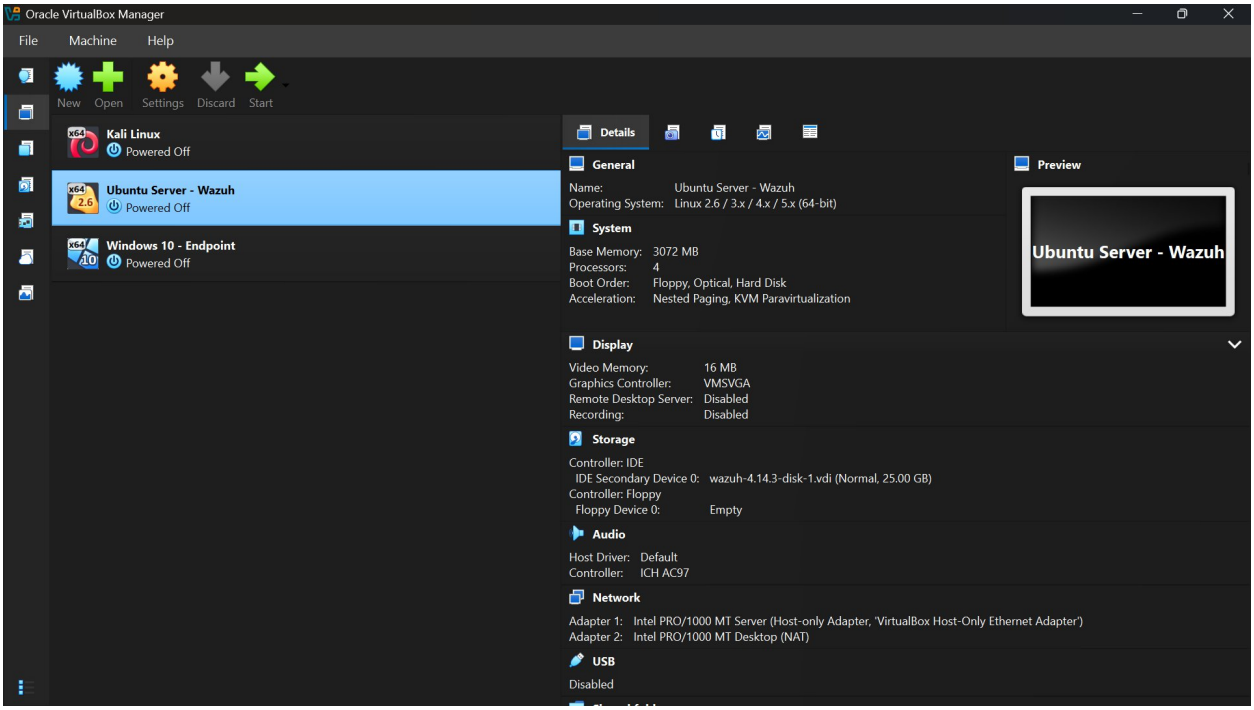
I deployed the three required VMs (Kali, Ubuntu, Windows 10) in VirtualBox. To ensure the lab remains isolated but functional, I configured a dual-adapter setup for each machine:

- **Adapter 1 (Host-Only Network):** Assigned to the 192.168.56.x subnet. This provides a secure, internal network allowing the VMs to communicate directly with the Wazuh Manager without exposing lab traffic to the external network.
- **Adapter 2 (NAT Network):** Assigned to the 10.0.x.x range. This allows the VMs to reach the outside internet to download necessary updates and agent installers.

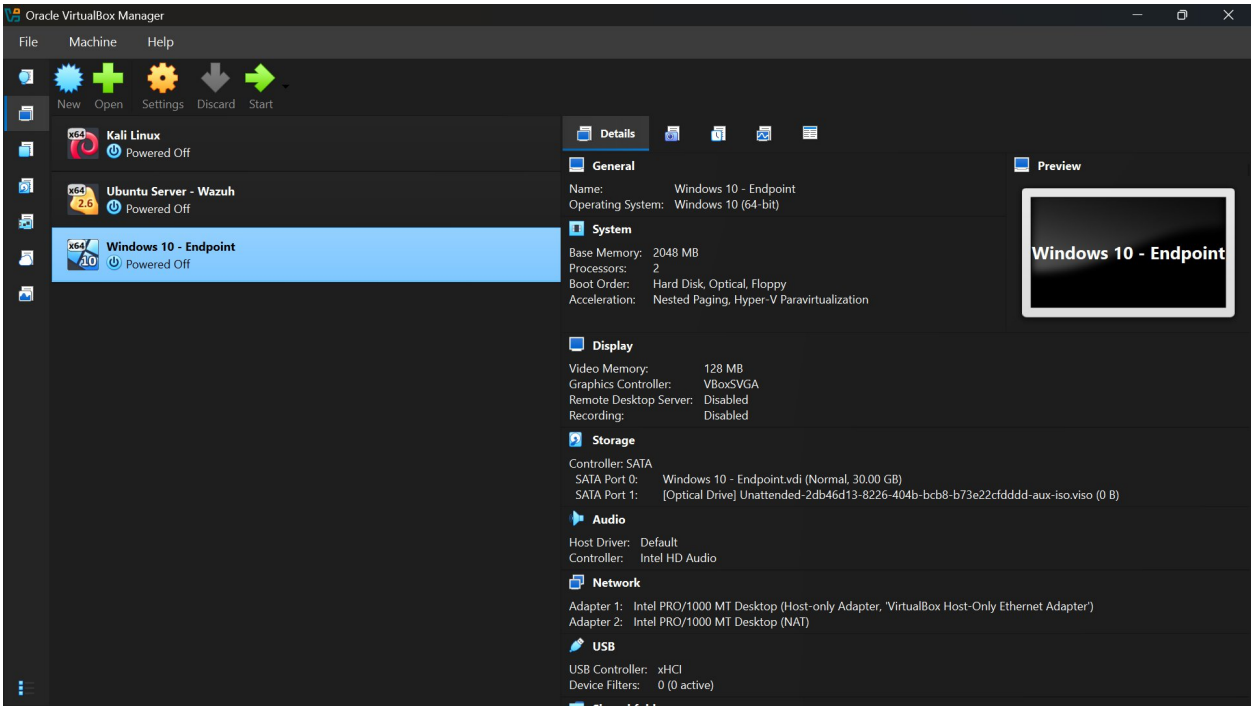
KALI LINUX



UBUNTU



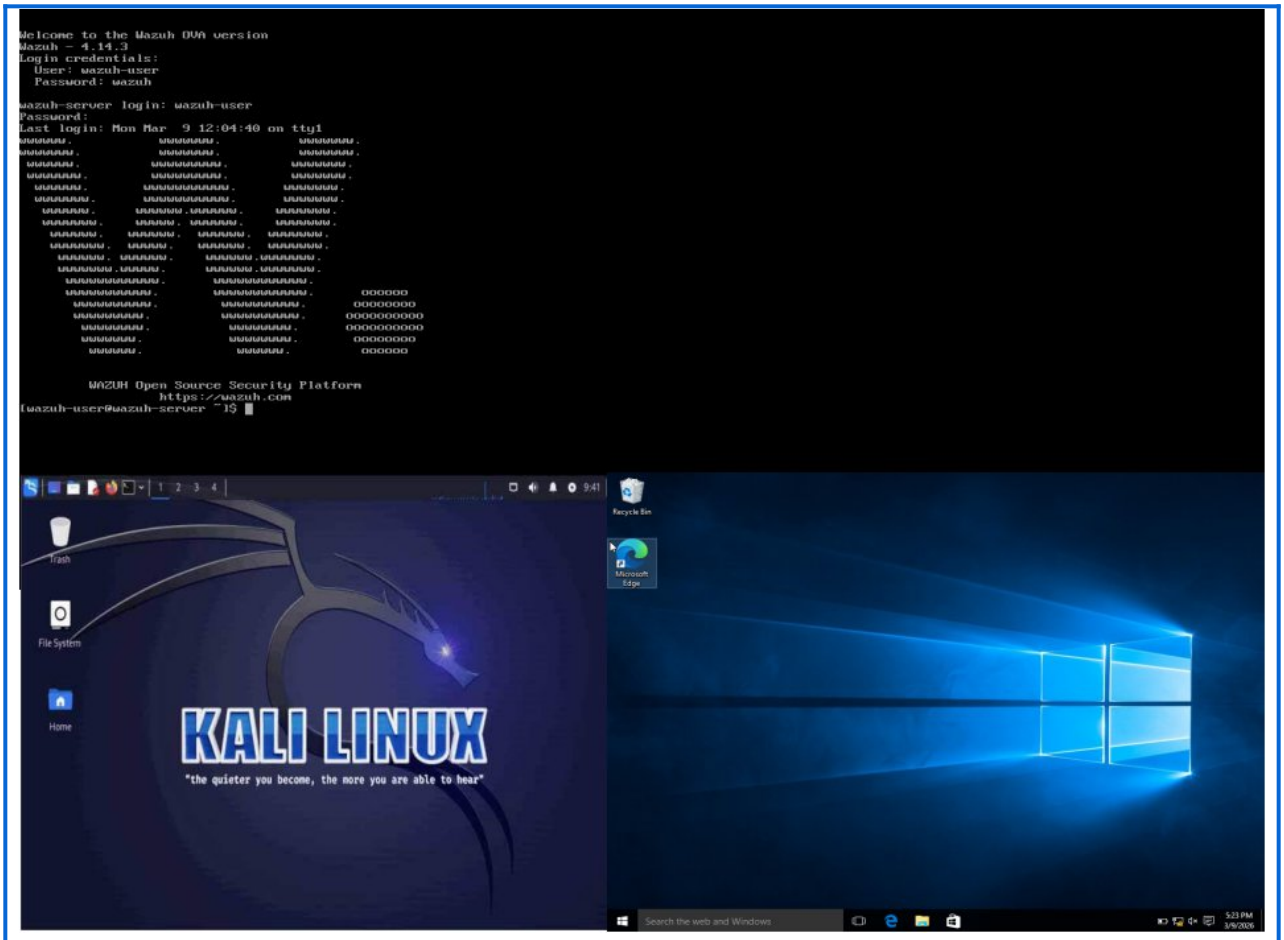
WINDOWS



Task 02: Hardware Constraints & Architecture Pivot

During the deployment phase, I encountered resource limitations on my host machine. The standard Wazuh installation script requires a minimum of 4GB (preferably 8GB) of RAM allocated to the Ubuntu Server to prevent the Wazuh Indexer from failing. Running this alongside Kali and Windows simultaneously exceeded my host's physical RAM capacity.

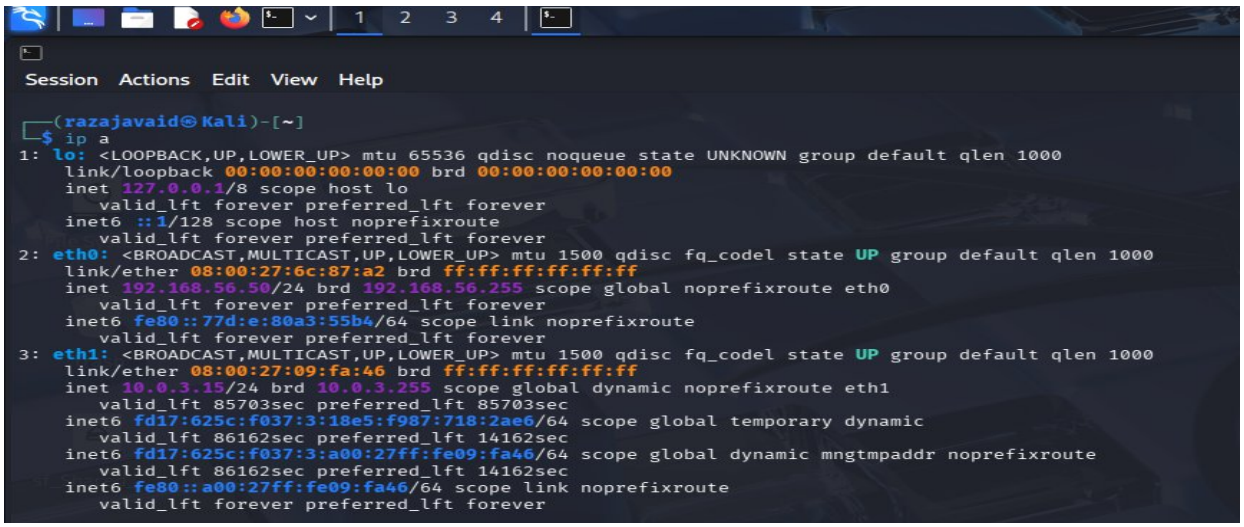
Solution: To adapt to this hardware limitation, I pivoted from the manual installation script to deploying the pre-configured **Wazuh OVA** (Open Virtual Appliance). This provided an optimized Wazuh Manager and Indexer out of the box, bypassing the hardware bottleneck while achieving the exact same architectural goal. I utilized composite screenshotting to document the functional state of all VMs.



Task 03: IP Configuration and Connectivity Verification

I verified the IP addresses for the machines, ensuring they successfully pulled IPs from both the Host-Only and NAT DHCP servers.

KALI LINUX



UBUNTU

```
[wazuh-user@wazuh-server ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:92:0b:19 brd ff:ff:ff:ff:ff:ff
    altname enp0s8
    inet 10.0.3.15/24 metric 1024 brd 10.0.3.255 scope global dynamic eth0
       valid_lft 86350sec preferred_lft 86350sec
    inet6 fd17:625c:f037:3:a00:27ff:fe92:b19/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86352sec preferred_lft 14352sec
    inet6 fe80::a00:27ff:fe92:b19/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c8:ae:22 brd ff:ff:ff:ff:ff:ff
    altname enp0s17
    inet 192.168.56.104/24 metric 1024 brd 192.168.56.255 scope global dynamic eth1
       valid_lft 550sec preferred_lft 550sec
    inet6 fe80::a00:27ff:fec8:ae22/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
[wazuh-user@wazuh-server ~]$
```

WINDOWS

```
Command Prompt

C:\Users\vboxuser>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::24d9:b031:d442:9dfb%4
   IPv4 Address. . . . . . . . . . . : 192.168.56.105
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : fd17:625c:f037:3:3580:5445:c5ff:a383
   Temporary IPv6 Address. . . . . . : fd17:625c:f037:3:284d:dc68:4b26:9adc
   Link-local IPv6 Address . . . . . : fe80::3580:5445:c5ff:a383%2
   IPv4 Address. . . . . . . . . . . : 10.0.3.15
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::2%2
                                       10.0.3.2

Tunnel adapter isatap.{6F86AC31-5B99-43B0-8EC0-B0AC3E6AED76}:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

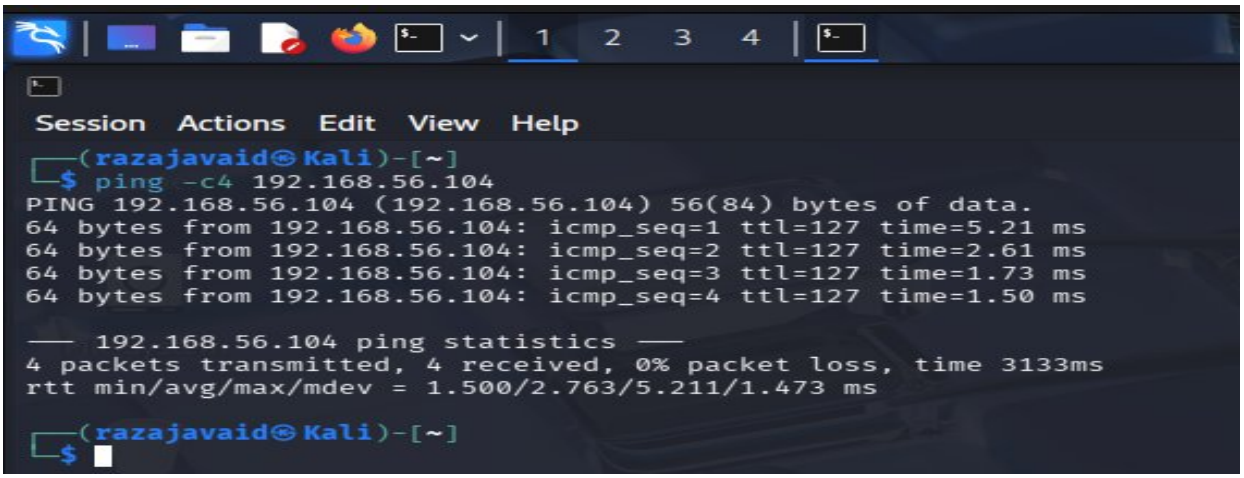
Tunnel adapter isatap.{17A88218-C080-4051-B575-E96890F1E317}:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

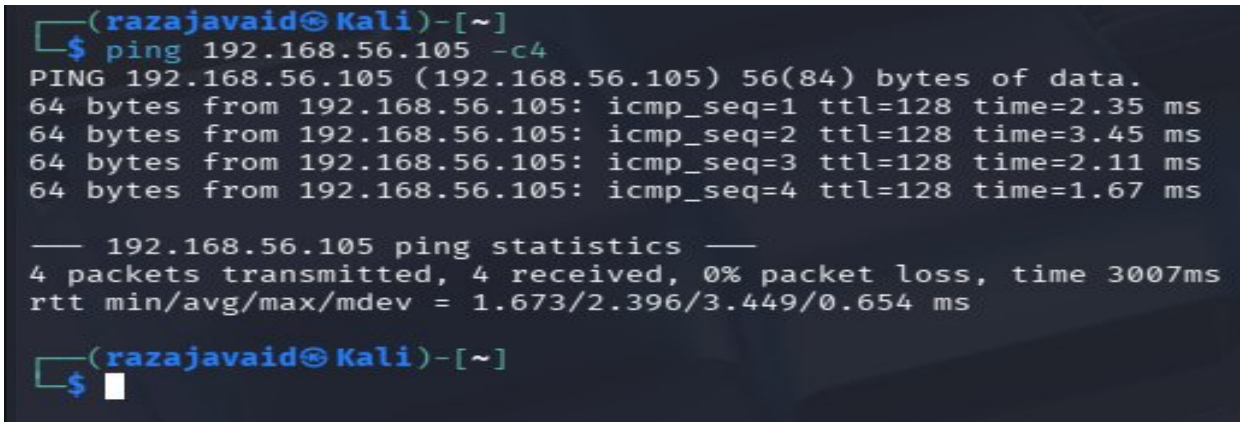
Next, I ran connectivity tests to confirm that all internal lab routing and external internet access were functioning correctly.

Internal Lab Connectivity Tests:

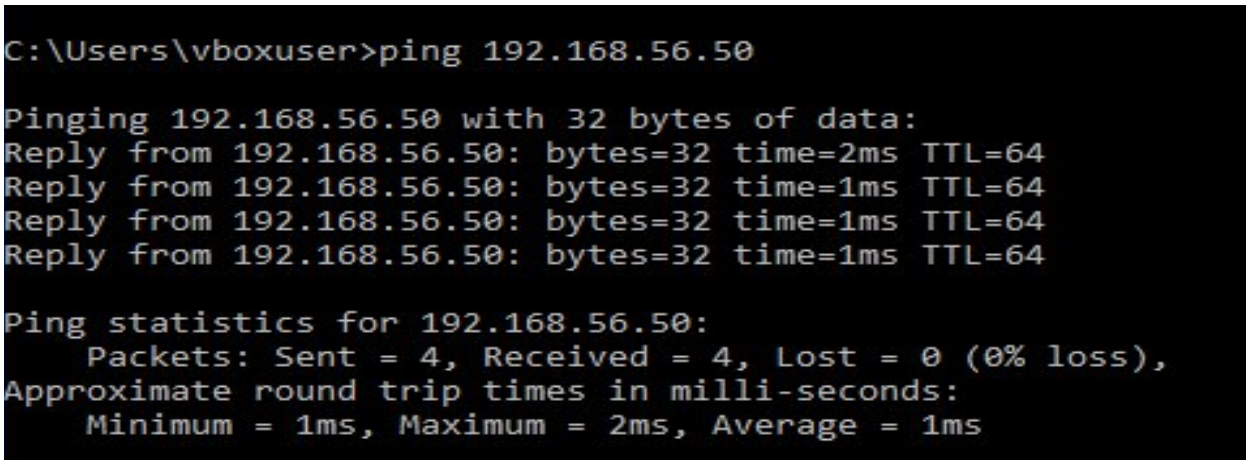
KALI LINUX TO UBUNTU



KALI LINUX TO WINDOWS



WINDOWS TO KALI LINUX



WINDOWS TO UBUNTU

```
C:\Windows\system32>ping 192.168.56.104

Pinging 192.168.56.104 with 32 bytes of data:
Reply from 192.168.56.104: bytes=32 time=4ms TTL=127
Reply from 192.168.56.104: bytes=32 time=2ms TTL=127
Reply from 192.168.56.104: bytes=32 time=3ms TTL=127
Reply from 192.168.56.104: bytes=32 time=2ms TTL=127

Ping statistics for 192.168.56.104:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
Approximate round trip times in milli-seconds:
    Minimum = 2ms, Maximum = 4ms, Average = 2ms
```

UBUNTU TO WINDOWS

```
[wazuh-user@wazuh-server ~]$ ping 192.168.56.105
PING 192.168.56.105 (192.168.56.105) 56(84) bytes of data.
64 bytes from 192.168.56.105: icmp_seq=1 ttl=128 time=2.49 ms
64 bytes from 192.168.56.105: icmp_seq=2 ttl=128 time=1.39 ms
64 bytes from 192.168.56.105: icmp_seq=3 ttl=128 time=2.29 ms
64 bytes from 192.168.56.105: icmp_seq=4 ttl=128 time=1.86 ms
^C
--- 192.168.56.105 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.393/2.009/2.489/0.421 ms
[wazuh-user@wazuh-server ~]$
```

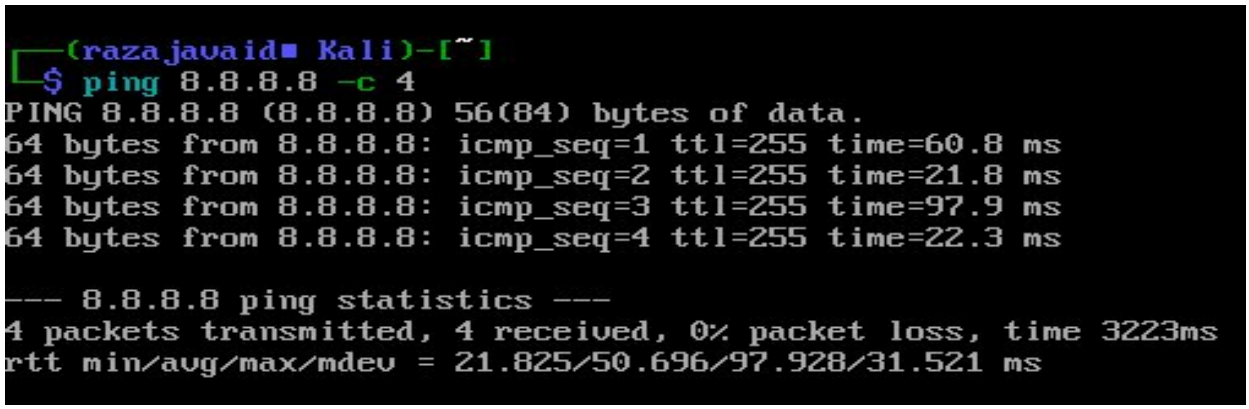
UBUNTU TO KALI LINUX

```
[wazuh-user@wazuh-server ~]$ ping -c4 192.168.56.50
PING 192.168.56.50 (192.168.56.50) 56(84) bytes of data.
64 bytes from 192.168.56.50: icmp_seq=1 ttl=64 time=11.0 ms
64 bytes from 192.168.56.50: icmp_seq=2 ttl=64 time=2.87 ms
64 bytes from 192.168.56.50: icmp_seq=3 ttl=64 time=2.82 ms
64 bytes from 192.168.56.50: icmp_seq=4 ttl=64 time=1.18 ms

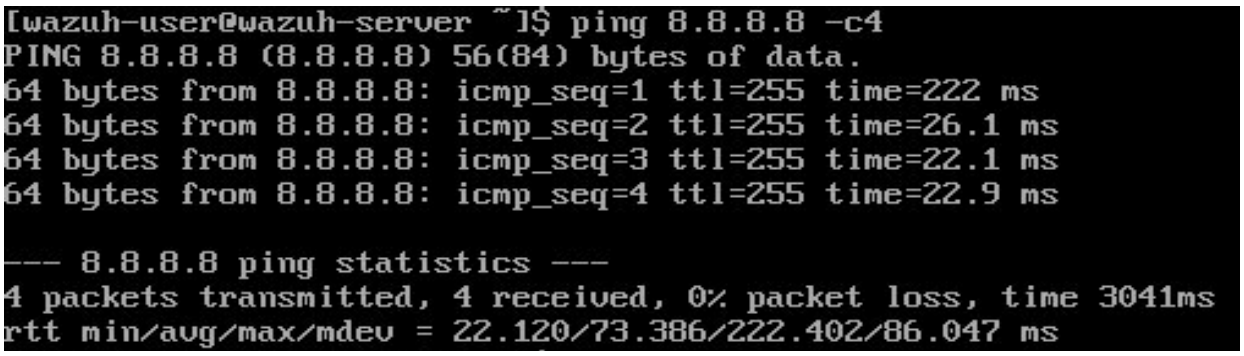
--- 192.168.56.50 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.182/4.458/10.966/3.817 ms
```

External Internet Connectivity Tests (NAT Verification):

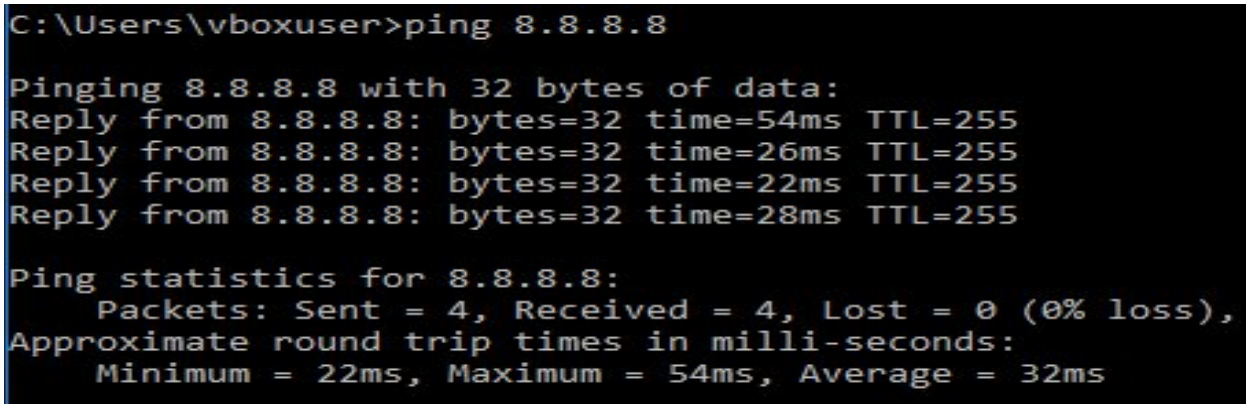
KALI LINUX



UBUNTU



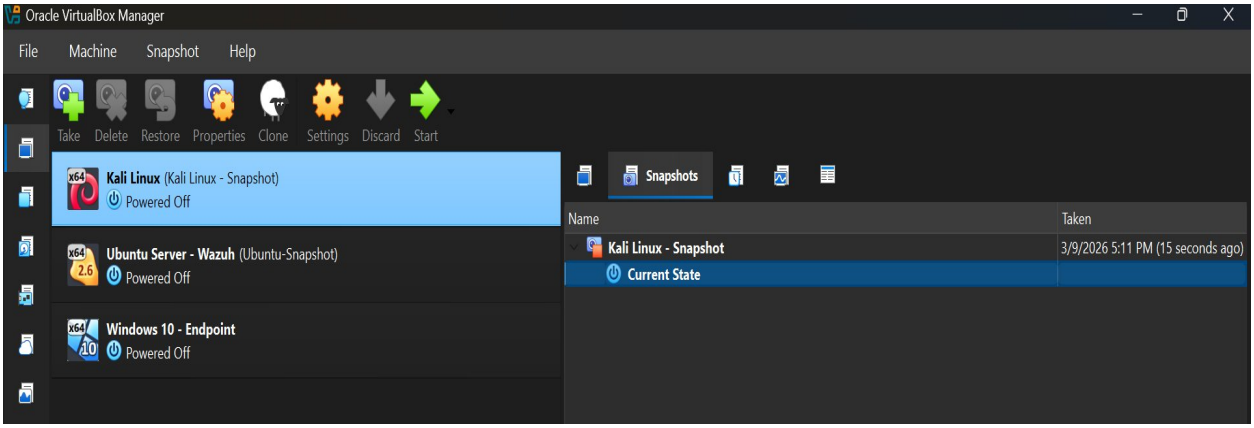
WINDOWS



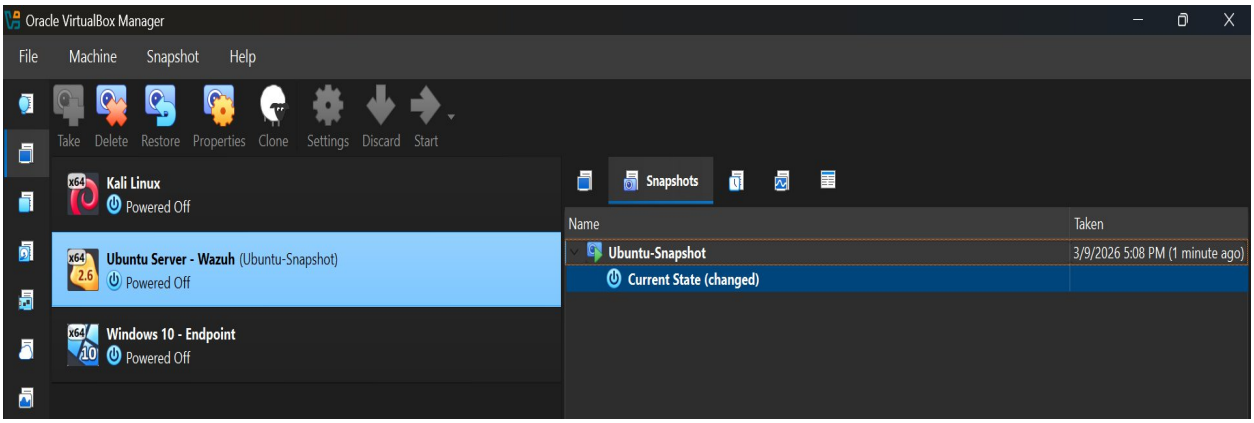
Task 04: VM Snapshots

With the network fully configured and tested, I took secure snapshots of all three machines. This established a safe rollback point before altering system files during agent deployment.

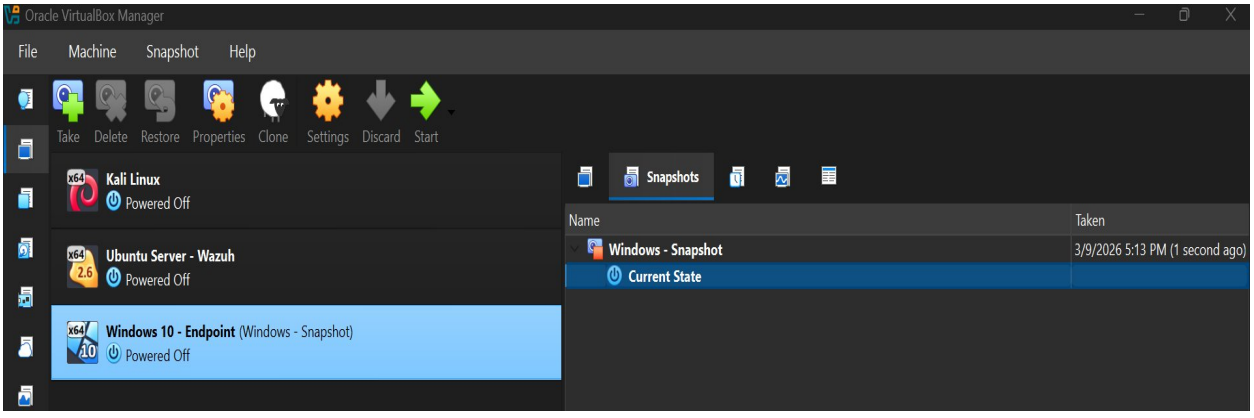
KALI LINUX



UBUNTU



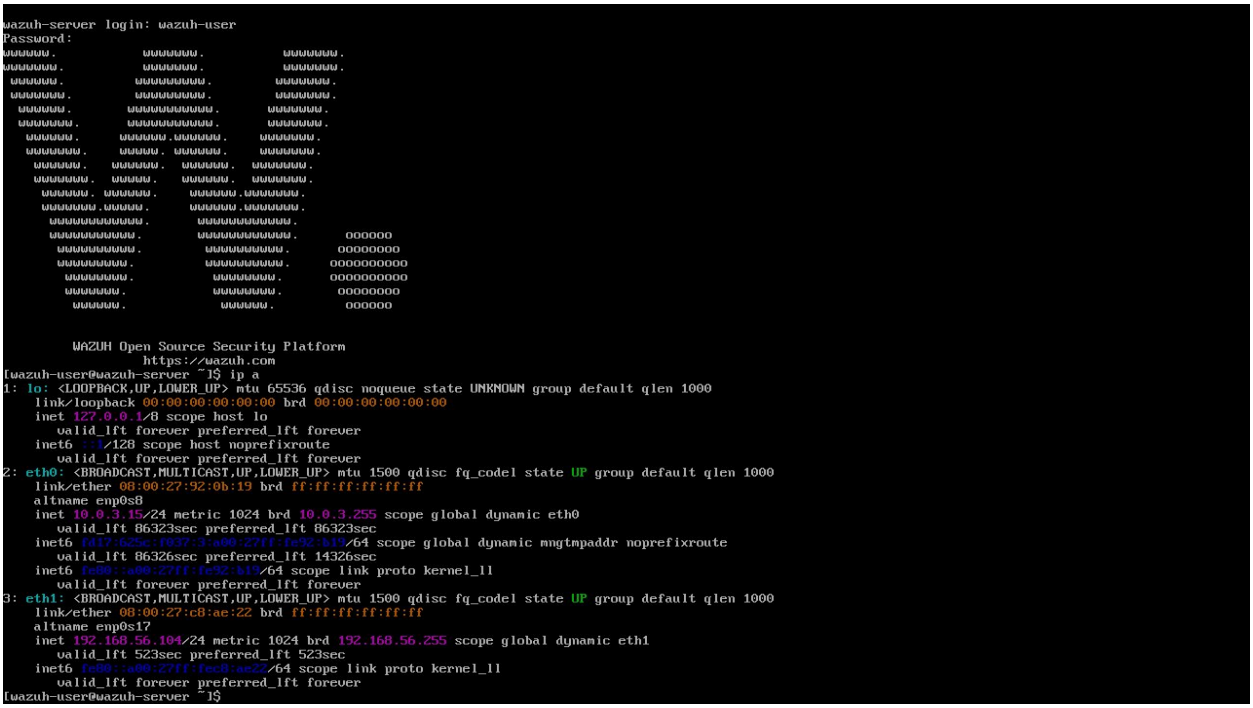
WINDOWS



DAYS 3 & 4: WAZUH MANAGER INSTALLATION

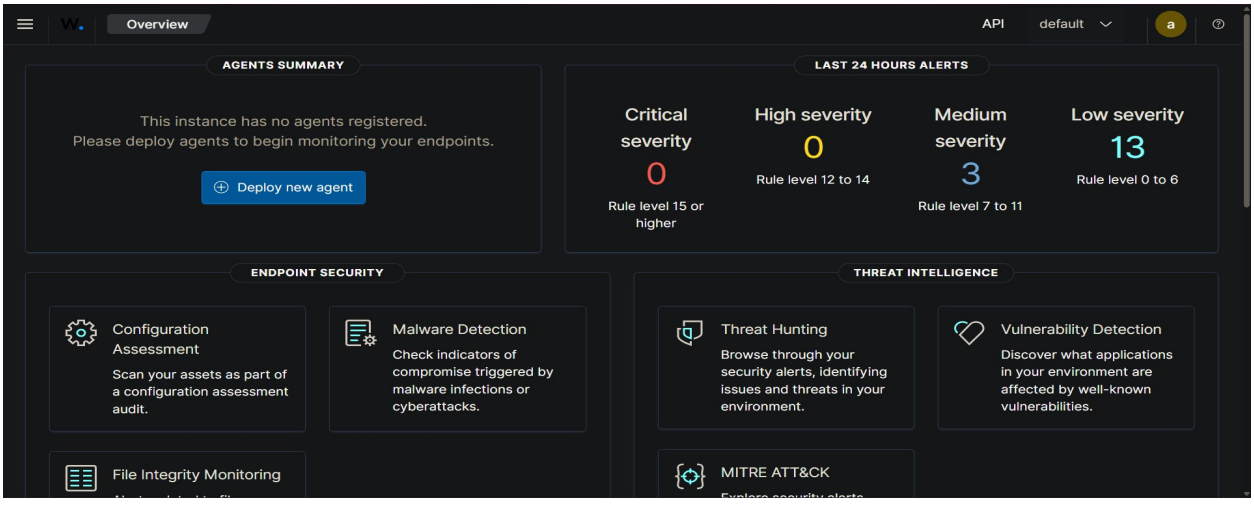
Task 01: Wazuh Configuration

As noted in Day 2, I successfully deployed the Wazuh OVA to manage system resources. I assigned a static IP address (192.168.56.104) to the Ubuntu Server on the Host-Only interface. This step is critical to ensure that endpoint agents do not lose connection to the manager if a DHCP lease expires.



Task 02: Accessing the Dashboard

Once the Wazuh Manager, Indexer, and Dashboard services initialized, I accessed the web interface via the host browser, confirming the SIEM was active and ready to receive telemetry.



DAYS 5 & 6: WAZUH AGENT DEPLOYMENT

Task 01: Deploying Agents to Endpoints

To gain visibility into the network, I deployed Wazuh Agents to both the Kali Linux and Windows 10 endpoints. I used the Wazuh Dashboard deployment wizard to generate the installation commands, pointing them to the static IP of the Wazuh Manager (192.168.56.104).

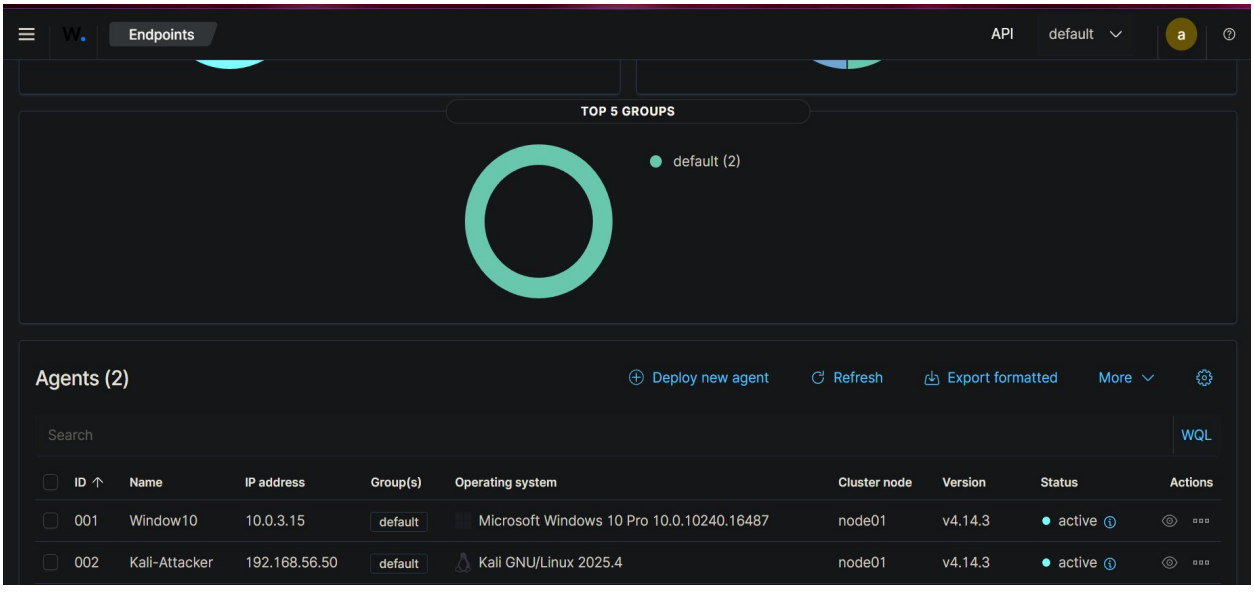
After installing the packages, I verified the ossec.conf file on the Kali machine to ensure the <client> block was correctly pointing to the manager.

```
┌──(razajavaid㉿kali)-[~]
└─$ sudo head -n 30 /var/ossec/etc/ossec.conf
[sudo] password for razajavaid:
<!--
  Wazuh - Agent - Default configuration for kali 2025.4
  More info at: https://documentation.wazuh.com
  Mailing list: https://groups.google.com/forum/#!forum/wazuh
-->

<ossec_config>
  <client>
    <server>
      <address>192.168.56.104</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
    <config-profile>kali, kali2025, kali2025.4</config-profile>
    <notify_time>20</notify_time>
    <time-reconnect>60</time-reconnect>
    <auto_restart>yes</auto_restart>
    <crypto_method>aes</crypto_method>
    <enrollment>
      <enabled>yes</enabled>
      <agent_name>Kali-Attacker</agent_name>
      <authorization_pass_path>etc/authd.pass</authorization_pass_path>
    </enrollment>
  </client>

  <client_buffer>
    <!-- Agent buffer options -->
    <disabled>no</disabled>
    <queue_size>5000</queue_size>
    <events_per_second>500</events_per_second>
```

After starting the agent services, both endpoints successfully registered and appeared as "Active" in the Wazuh dashboard.



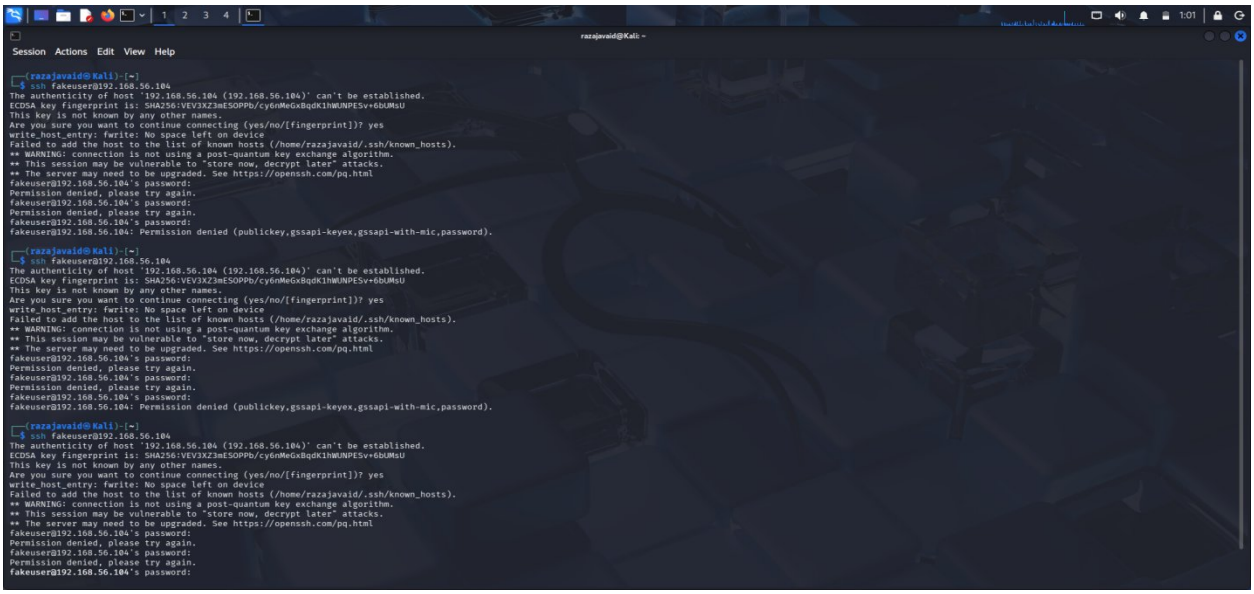
Task 02: Generating Test Alerts (Threat Simulation)

To confirm the data pipeline was actively processing logs and triggering rules, I simulated attacks and system changes against the environment.

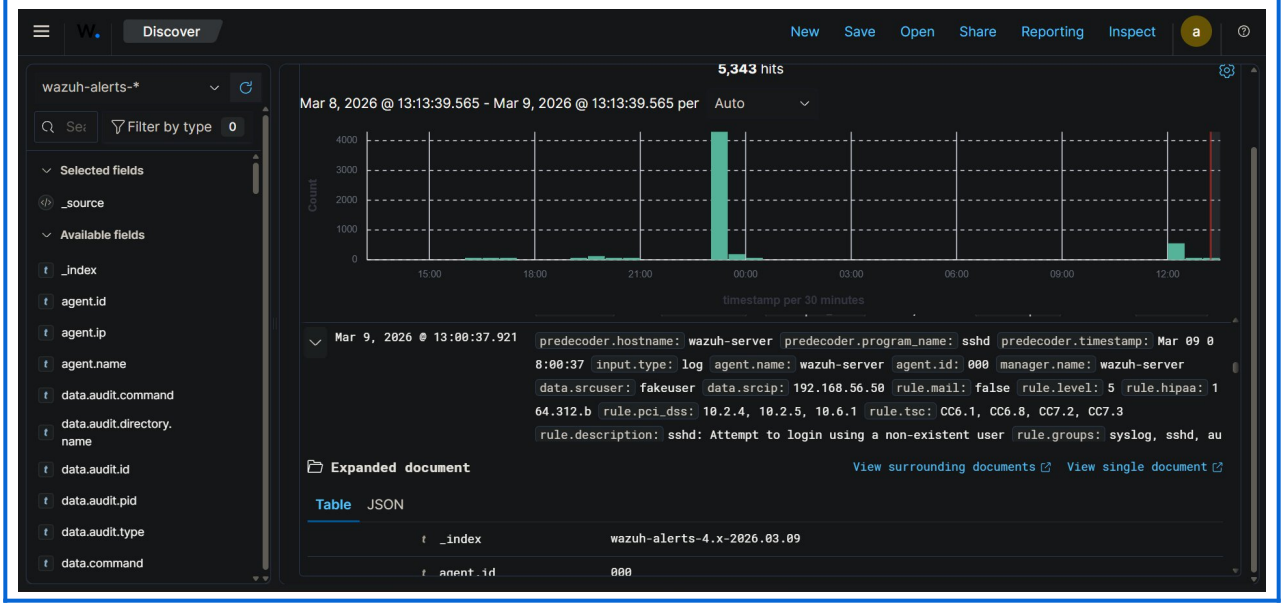
Simulation 1: SSH Brute Force (Network Attack)

From the Kali Linux machine, I intentionally spammed failed SSH login attempts against a non-existent user on the Ubuntu Server to trigger an authentication alert.

SSH BRUTE FORCE ATTACK



WAZUH CATCHES SSH ATTACK



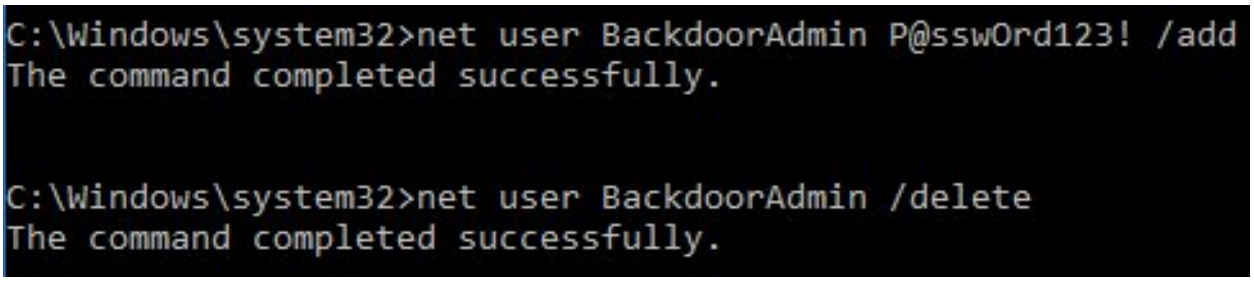
WAZUH SSH ALERT



Simulation 2: Privilege Escalation / Backdoor Creation (Endpoint Attack)

On the Windows 10 machine, I opened an elevated command prompt and executed net user BackdoorAdmin P@ssw0rd123! /add to simulate an attacker establishing persistence. Wazuh successfully caught the user creation event.

WINDOWS BACKDOOR TEST



WINDOWS ALERT CAUGHT



Simulation 3: PowerShell Telemetry Testing

To verify extended visibility, I executed PowerShell commands on the endpoint and observed the corresponding telemetry populated in the SIEM.

POWERSHELL TELEMETRY TEST

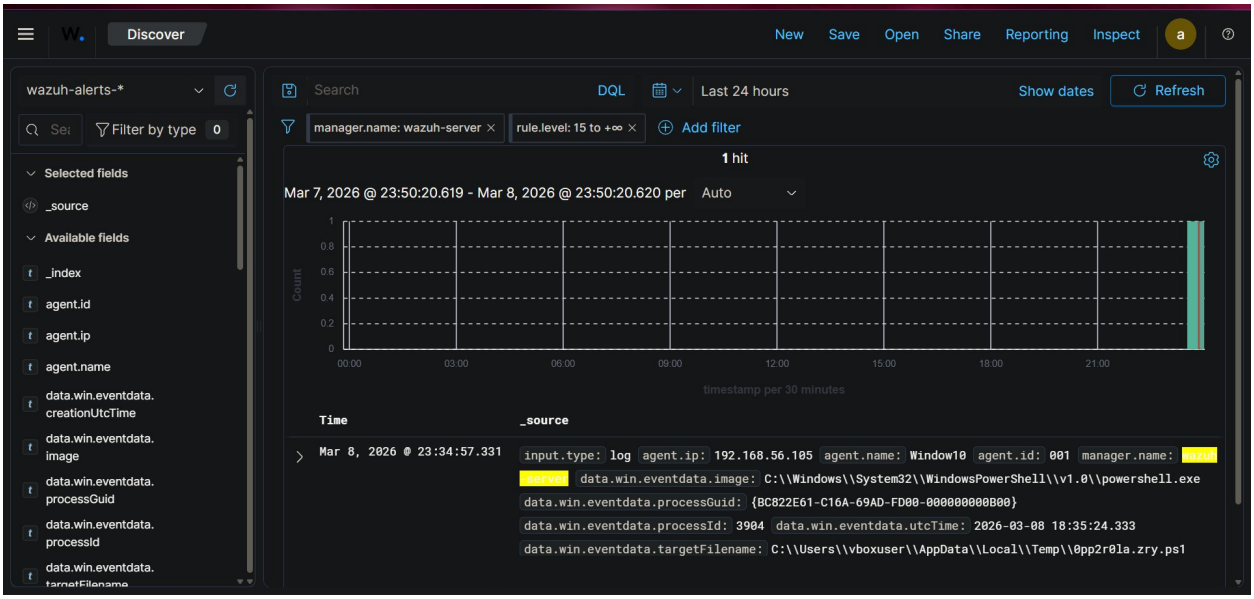
```
C:\Windows\system32>powershell.exe -c "Test-NetConnection -ComputerName 8.8.8.8 -Port 53"


ComputerName            : 8.8.8.8
RemoteAddress           : 8.8.8.8
RemotePort              : 53
InterfaceAlias          : Ethernet 2
SourceAddress           : 10.0.3.15
PingSucceeded           : True
PingReplyDetails (RTT) : 21 ms
TcpTestSucceeded        : True
```

TELEMETRY ALERT CAUGHT



TELEMETRY ALERT DETAILS



DAY 7: DASHBOARD NAVIGATION & ALERT TRIAGE

Task 01: Exploring the SIEM Modules

I utilized the dashboard to shift from deployment to active analysis, focusing heavily on the Threat Hunting and MITRE ATT&CK modules to map the generated alerts to industry-standard adversary tactics.



Task 02: Environmental Baseline (Top 10 Alerts)

To understand what "normal" looks like in my specific lab environment, I identified the top 10 most frequently occurring alert types over the last 24 hours.

Rank	Rule ID	Level	Rule Description	Assessment
1	92217	6	Executable dropped in Windows root folder	**Routine Activity:** Due to recent VM setup and software installations, a high volume of dropped executables is expected baseline noise.
2	80730	3	Auditd: SELinux permission check	**Routine Activity:** Standard background telemetry generated by the Linux kernel enforcing permissions securely.
3	510	7	Host-based anomaly detection event (rootcheck)	**Routine Activity:** Indicates the Wazuh agent is successfully performing its scheduled baseline rootkit scans.
4	5501	3	PAM: Login session opened	**Routine Activity:**

				Expected administrative behavior as I am actively logging into the Linux machines to configure them.
5	5716	5	sshd: Attempt to login using a non-existent user	**Genuine Concern (Simulated):** Caught my intentional SSH brute-force simulation from Kali.
6	60790	4	The registry file is still in use by other applications	**Routine Activity:** Standard Windows OS behavior occurring during normal background service operations.
7	60137	3	Windows User Logoff	**Routine Activity:** Standard audit logging confirming a user session has ended.
8	92219	6	Possible DLL search order hijack	**False Positive:** Triggered by wuapi.dll, which is the built-in Windows

				Update service staging routine patches.
9	92021	3	Powershell was used to delete files or directories	**Routine Activity:** Likely an automated background cleanup script running after Windows updates in this fresh VM.
10	60646	5	License activation (slui.exe) failed	**Routine Activity:** The temporary Windows lab VM lacks a paid license key, generating errors with zero security impact.

Task 03: Critical Alert Triage (Ransomware Simulation)

To push the Threat Hunting module to its limits, I executed a simulated Ransomware command on the Windows 10 endpoint: vssadmin delete shadows /all /quiet. Ransomware strains commonly use this command to delete volume shadow copies so victims cannot restore their files.

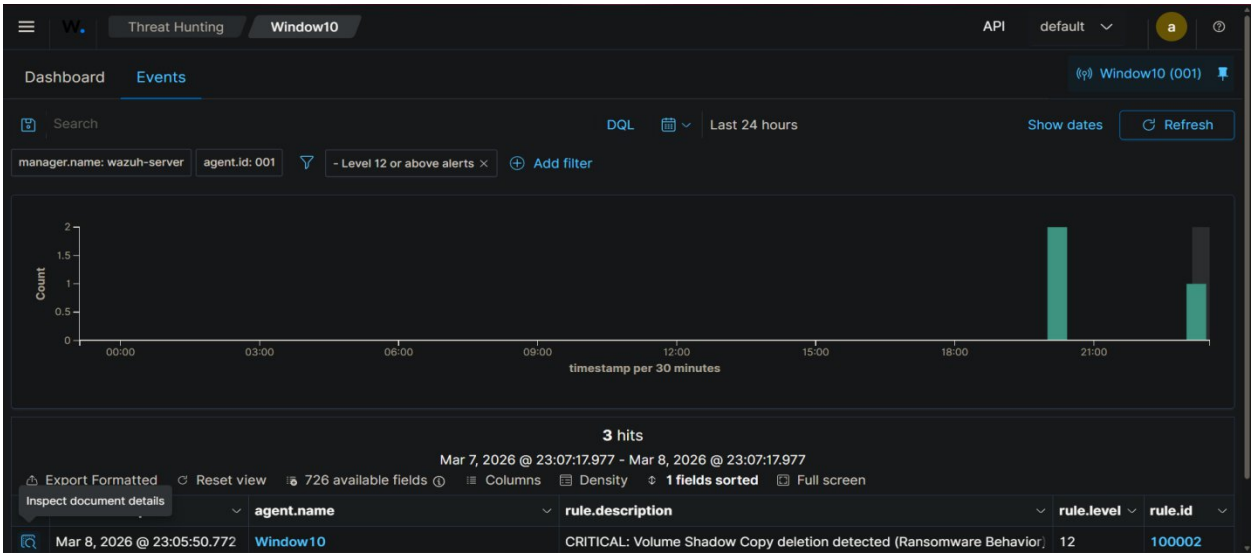
Wazuh instantly caught this telemetry via Sysmon and categorized it as a **Level 12 CRITICAL** alert.

SIMULATED RANSOMWARE EXECUTION

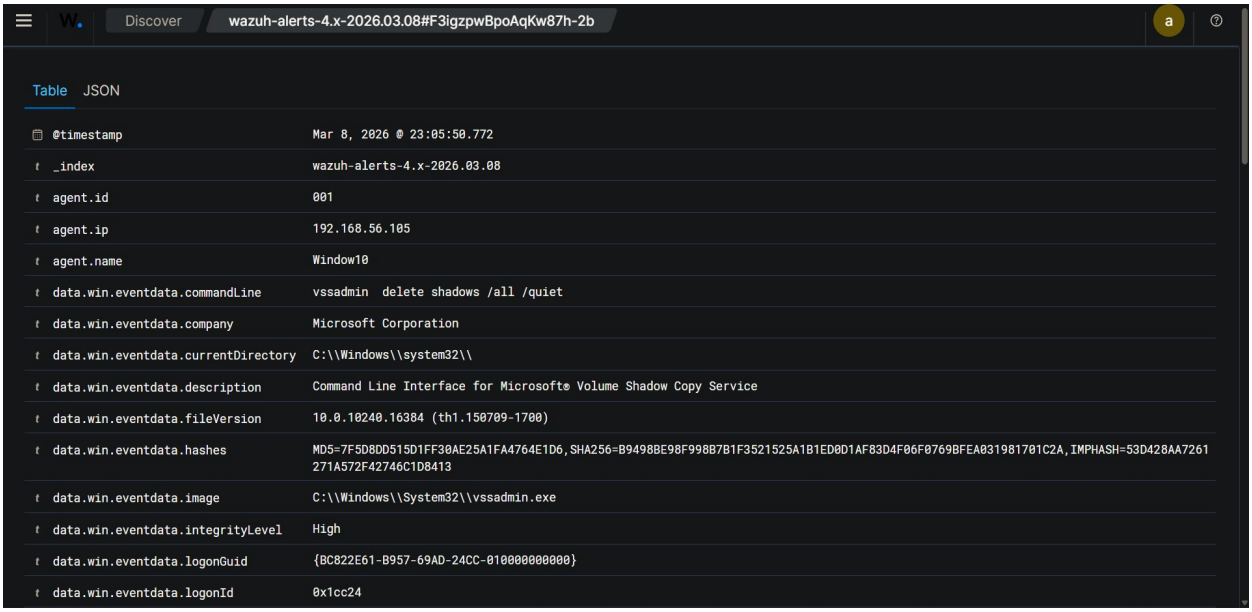
```
C:\Windows\system32>vssadmin delete shadows /all /quiet
vssadmin 1.1 - Volume Shadow Copy Service administrative command-line tool
(C) Copyright 2001-2013 Microsoft Corp.

No items found that satisfy the query.
```

RANSOMWARE CRITICAL ERROR



RANSOMWARE ALERT DETAILS



Triage Analysis:

- **What is the alert?** Rule 100002 triggered, detecting the specific command line arguments used to delete shadow volume copies.
- **Is it a false positive?** No, it is a True Positive catching my simulated attack.
- **Action Required:** In a production environment, this would warrant immediate isolation of the Windows host from the network and an incident response investigation into how the attacker gained code execution.

End of Week 1 Report